

PROJECT TITLE

WATCH MY MOVIE

Movie
Recommendation
System

Name: Manasvi Jain

Reg No.: 26BCE11247

Branch: BTech
Computer Science

**College: VIT Bhopal
University**

Date: 29-09-2026

INTRODUCTION

The Watch My Movie project is a Python-based command-line application designed to help users discover movies from a movie dataset.

The system provides three main functions: recommending movies

based on selected genres, finding information about a particular movie, and recommending movies based on a specified rating.

The project uses Pandas to load and filter the movie dataset and CSV files to store user-related records and generated results.

PROBLEM STATEMENT

- Users may find it difficult to choose suitable movies from a large collection.
- Searching manually for movies belonging to multiple genres can be time-consuming.
- Users may want quick information about a

particular movie,
including its genres
and rating.

- The project applies Python programming concepts to create a menu-driven movie assistance system.

FUNCTIONAL REQUIREMENTS

- Load movie information from

movies_dataset.csv
using Pandas.

- Accept the user's name and movie-related preferences.
- Recommend movies matching one, two, or three selected genres.
- Search for a movie by name and display its genres and average

rating.

- Recommend movies whose rating matches the rating entered by the user.
- Store records in CSV files.
- Display appropriate messages for invalid input or missing movies.

NON-FUNCTIONAL REQUIREMENTS

- Performance: Dataset filtering should execute quickly.
- Reliability: Invalid menu input should not crash the program.
- Usability: The application provides a simple numbered menu.

- Portability: The project uses Python and common libraries.
- Maintainability: Separate functions keep tasks organised.

SYSTEM ARCHITECTURE

The project follows a simple

Input-Process-Output structure.

Data Layer:

movies_dataset.csv
contains movie
information.

Processing Layer:

Python functions use
Pandas filtering and
conditional logic.

Storage Layer: CSV files
store user searches and
recommendation
records.

Presentation Layer: A command-line interface displays menus and results.

DESIGN DECISIONS

- Python was selected for simple syntax and data-processing support.
- Pandas was selected for CSV loading and DataFrame filtering.

- CSV files were selected for simple persistent storage.
- A command-line interface focuses on programming and data-processing functionality.
- Separate functions keep different tasks organised.

IMPLEMENTATION DETAILS

The program reads `movies_dataset.csv` using `pd.read_csv()`. The main menu asks the user to choose one of three operations.

The genre recommendation function accepts one, two, or three genres and

filters the dataset using Pandas.

The movie information function searches Movie_Name and displays genre and rating information.

The rating recommendation function compares the entered rating with the Rating column.

CSV files are used to
append records for
searches and
recommendations.

CODE SCREENSHOTS

Code Screenshot 1

```
import pandas as pd
import csv
df=pd.read_csv("movies_dataset.csv",index_col=0)
def genrefinder(name,moviedata):
    data=[name]+moviedata
    f=open("genrefound.csv","a",newline="")
    w=csv.writer(f)
    w.writerow(data)
    f.close()
def storerecords(name,genres):
    data=[name,genres]
    f=open("records.csv","a",newline="")
    w=csv.writer(f)
    w.writerow(data)
    f.close()
def storerates(name,rate):
    data=[name,rate]
    f=open("rates.csv","a",newline="")
    w=csv.writer(f)
    w.writerow(data)
    f.close()
def movierecgenre():
    name=input("enter your name : ")
    g1=input("enter the first genre : ")
    check=df['Genre'].str.contains(g1,case=False,na=False)
    ch=input("do you want to add second genre (y/n)? ")
    i=2
    genres=[g1]
    while (ch=="y"):
```

Code Screenshot 2

```
while (ch=="y"):  
    if i==2:  
        g2=input("enter the second genre : ")  
        check=check & df['Genre'].str.contains(g2,case=False,na=False)  
        genres.append(g2)  
    else:  
        g3=input("enter the third genre : ")  
        check=check & df['Genre'].str.contains(g3,case=False,na=False)  
        genres.append(g3)  
    if i==3:  
        break  
    i+=1  
    ch=input("do you want to add third genre (y/n)? ")  
filtered=df[check]  
if filtered.empty:  
    print("no movie found sorry")  
else:  
    print("RECCOMENDATIONS")  
    r=filtered['Movie_Name'].unique()[:30]  
    for movie in r:  
        print(movie)  
storerecords(name,genres)  
print("THANK YOU")  
  
def genratefinder():  
    name=input("enter your name : ")  
    movie=input("enter movie name: ")  
    check=df['Movie_Name'].str.contains(movie,case=False,na=False)  
    filtered=df[check]
```

Code Screenshot 3

```
def movierecrate():
    name=input("enter your name : ")
    rate=float(input("enter rating: "))
    check=df['Rating']==rate
    filtered=df[check]
    r=filtered['Movie_Name'].unique()[:30]
    for movie in r:
        print(movie)
    storerates(name,rate)
    print("THANK YOU")
print("      WELCOME TO WATCH MY MOVIE!      ")
print("      How Can I Help You Today?      ")
print("      1:recommend movies based on genres      ")
print("      2:more about this movie      ")
print("      3:recommend movies based on ratings      ")
try:
    ch=int(input("      PLEASE ENTER YOUR CHOICE (1/2/3): "))
    if ch==1:
        movierecgenre()
    elif ch==2:
        genratefinder()
    elif ch==3:
        movierecrate()
except ValueError:
    print("sorry invalid value please enter a valid value")
|
```

OUTPUT SCREENSHOTS

Output Screenshot 1

```
Python 3.13.7 (v3.13.7:bceelc32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (c
ang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/manasvijain/Desktop/projectclg/finalclg.py =====
WELCOME TO WATCH MY MOVIE!
How Can I Help You Today?
1:recommend movies based on genres
2:more about this movie
3:recommend movies based on ratings
PLEASE ENTER YOUR CHOICE (1/2/3): 1
enter your name : manasvi
enter the first genre : mystery
do you want to add second genre (y/n)? y
enter the second genre : horror
do you want to add third genre (y/n)? y
enter the third genre : crime
RECCOMENDATIONS
American Psycho (2000)
Identity (2003)
Copycat (1995)
Death Note (2006)
Paranoia Agent (2004)
From Hell (2001)
American Psycho II: All American Girl (2002)
Saw VI (2009)
Velvet Buzzsaw (2019)
Book of Shadows: Blair Witch 2 (2000)
```

Output Screenshot 2

Velvet Buzzsaw (2019)
Book of Shadows: Blair Witch 2 (2000)
Mindhunters (2004)
Horsemen (2009)
Don't Click (2012)
Hound of the Baskervilles, The (1959)
Rubber (2010)
House of Wax (1953)
God told Me To (1976)
Testament of Dr. Mabuse, The (Das Testament des Dr. Mabuse) (1933)
Bird with the Crystal Plumage, The (Uccello dalle piume di cristallo, L') (1970)
Murders in the Rue Morgue (1932)
The Hound of the Baskervilles (1988)
Gozu (Gokudô kyôfu dai-gekijô: Gozu) (2003)
Black Butler (2014)
1000 Eyes of Dr. Mabuse, The (Die 1000 Augen des Dr. Mabuse) (1960)
Last Will of Dr. Mabuse, The (Testament du Dr. Mabuse, Le) (1933)
The Killing Floor (2007)
New York Ripper, The (Squartatore di New York, Lo) (1982)
8213: Gacy House (2010)
Opera (1987)
Jack's Back (1988)
THANK YOU

Output Screenshot 3

```
===== RESTART: /Users/manasvijain/Desktop/projectclg/finalclg.py =====  
WELCOME TO WATCH MY MOVIE!  
How Can I Help You Today?  
  1:recommend movies based on genres  
  2:more about this movie  
  3:recommend movies based on ratings  
PLEASE ENTER YOUR CHOICE (1/2/3): 2  
enter your name : manasvi  
enter movie name: crazy rich asians  
MOVIE FOUND  
GENRES  
Comedy  
3.5  
THANK YOU
```

Output Screenshot 4

```
===== RESTART: /Users/manasvijain/Desktop/projectcig/r1na1c1g.py =
WELCOME TO WATCH MY MOVIE!
How Can I Help You Today?
  1:recommend movies based on genres
  2:more about this movie
  3:recommend movies based on ratings
PLEASE ENTER YOUR CHOICE (1/2/3): 3
enter your name : manasvi
enter rating: 0.5
Piano Teacher, The (La pianiste) (2001)
Mr. Holland's Opus (1995)
Little Women (1994)
Tomorrow Never Dies (1997)
There's Something About Mary (1998)
Superman IV: The Quest for Peace (1987)
Inspector Gadget (1999)
Iron Giant, The (1999)
Who Framed Roger Rabbit? (1988)
Last Boy Scout, The (1991)
Cold Mountain (2003)
Love Song for Bobby Long, A (2004)
Crash (2004)
Wedding Crashers (2005)
40-Year-Old Virgin, The (2005)
Percy Jackson & the Olympians: The Lightning Thief (2010)
Last Airbender, The (2010)
```

Output Screenshot 5

Cold Mountain (2003)
Love Song for Bobby Long, A (2004)
Crash (2004)
Wedding Crashers (2005)
40-Year-Old Virgin, The (2005)
Percy Jackson & the Olympians: The Lightning Thief (2010)
Last Airbender, The (2010)
Cowboys & Aliens (2011)
Prometheus (2012)
Avengers: Infinity War - Part I (2018)
Brady Bunch Movie, The (1995)
Indian in the Cupboard, The (1995)
Batman & Robin (1997)
Blair Witch Project, The (1999)
One Hour Photo (2002)
Cat in the Hat, The (2003)
Notebook, The (2004)
xXx: State of the Union (2005)
Kicking & Screaming (2005)
Expelled: No Intelligence Allowed (2008)
THANK YOU

TESTING APPROACH

- Menu Testing: The three menu choices were tested through the command-line interface.
- Genre Testing: Multiple genres were entered and recommendations were displayed.

- **Movie Search Testing:**
A movie was searched and its information was displayed.
- **Rating Testing:** A rating was entered and matching movies were displayed.
- **Invalid Input Testing:**
ValueError handling was included for invalid menu input.

CHALLENGES FACED

- Handling multiple genre selections.
- Searching text values without depending on exact capitalization.
- Managing CSV files for persistent records.
- Handling invalid menu input.
- Keeping the program divided into

understandable
functions.

LEARNINGS AND KEY TAKEAWAYS

- Improved understanding of Python functions and modular programming.
- Learned how to load and filter CSV data using Pandas.

- Practiced loops, conditions, user input, and exception handling.
- Learned how to write records into CSV files.
- Understood how a simple command-line application can solve a practical problem.

**FUTURE
ENHANCEMENT**

- Develop a graphical or web-based user interface.
- Add fuzzy search for movie names.
- Add recommendations using multiple factors such as rating and genre.
- Allow users to view more detailed movie

information.

- Add persistent user profiles and personalised recommendation history.

REFERENCES

- Python documentation and course concepts.
- Pandas library concepts used for CSV loading and

DataFrame filtering.

- Course notes.
- Lab components.